2017 年 12 月
第 38 卷 第 12 期

计算机工程与设计
COMPUTER ENGINEERING AND DESIGN

Dec. 2017
Vol. 38 No. 12

开源软件可信性评价方法

孙 晶[1]，刘丽丽[2+]

（1. 北方工业大学 计算机学院，北京 100144；
2. 北方工业大学 大规模流数据集成与分析技术北京市重点实验室，北京 100144）

摘 要：为有效解决企业和用户对开源软件的可信性的评价，分析开源软件的特点，提出开源软件可信性模型，给出开源软件的可信属性及组件可信性定义。爬取缺陷跟踪系统中开源软件可信证据，评价开源软件在实时运行时的可信性；在此基础上，对可信属性之间的联系进行分析，采取可信转移概率算法，评价组件之间的可信性。通过火狐浏览器的分析与验证，验证了该模型和该算法的有效性。

关键词：开源软件；可信性模型；缺陷跟踪系统；可信证据；火狐浏览器

中图法分类号：TP311 **文献标识号**：A **文章编号**：1000-7024（2017）12-3272-07

doi：10.16208/j.issn1000-7024.2017.12.015

Methodologicalstudy of evaluating dependability of open source software

SUN Jing[1]，LIU Li-li[2+]

（1. Computer College，North China University of Technology，Beijing 100144，China；2. Beijing Key Laboratory on Data Integration and Analysis Technology of Large-Scale Stream，North China University of Technology，Beijing 100144，China）

Abstract：To effectively solve the evaluation of the credibility of enterprises and users to open-source software，the characteristics of the open source software were analyzed，an open source software credibility model was proposed，which gave the definition of the trusted attribute of open source software and the definition of the credibility of the open source software component. Open source software credible evidence in defect tracking system was crawled，so as to evaluate the credibility of open source software in real time. On this basis，an analysis was made on the relationship between the trusted attributes，using open source software trusted transition probability algorithm，the credibility of components was evaluated. Through the analysis and verification of the Firefox browser，the effectiveness of the model and algorithm was verified.

Key words：open source software；credibility model；defect tracking system；credible evidence；Firefox browser

0 引 言

软件的可信性是对软件提供服务的信任度。在并发量大、持续时间长的状态下获得的实际结果与预期结果总是相一致的软件称为可信软件。针对可信性的研究已经取得了很大的成果。陈火旺教授等[1]给出软件的高可信性质定义：软件的可信性是指软件系统需要满足的关键性质，软件一旦违背这些关键性质，会造成不可容忍的损失，称这些关键性质为高可信性质。软件的关键性质是指安全性（security）、可靠性（reliability）、防危性（safety）、可维护性（maintainability）、可用性（availability）等，软件的可信属性总是会涉及其中一个或多个。

关于开源软件可信性的研究，袁霖等[2]提出了基于互联网的开源软件自动化可信证据框架，分析了软件开发阶段人员特征和配置角色结构对软件可信性的影响，其中采用自动化的方式获取开源软件的可信证据，既避免了专家评估的不稳定性，也避免了可信证据的重复性。该研究对开源软件自动化获取可信证据取得了很大的成果，建立了部分可信证据与开源数据的映射关系，但并未给出开源软件的可信属性数学定义、开源软件的整体可信性的可量化的可信性结果、一个组件依赖或转移于其它组件概率的相关研究。本文将以缺陷跟踪系统为研究案例，通过分析研究开源

收稿日期：2016-09-13；**修订日期**：2017-09-21

基金项目：国家自然科学基金项目（61370051）

作者简介：孙晶（1968-），女，辽宁沈阳人，硕士，副教授，研究方向为软件工程、软件测试；＋通讯作者：刘丽丽（1990-），女，山西临汾人，硕士研究生，研究方向为软件测试。E-mail：409134674@qq.com

(C)1994-2024 China Academic Journal Electronic Publishing House. All rights reserved. http://www.cnki.net

软件可信证据与开源数据之间的关系、可信证据与可信属性之间的关系、可信证据在组件之间的转移关系，采用自动化技术采集可信证据，运用数学公式量化可信结果。

本文提出开源软件可信性模型，该技术通过软件组件的可信性及之间的转移概率得到开源软件实时运行中的可信性。

1　开源软件可信性相关概念

1.1　开源软件可信性模型概念

本文的开源软件可信性模型是建立在文献［2］提出的开源软件相关的可信证据基础之上，其中，本文以缺陷跟踪系统自动化获取的主要证据信息为例，分析了可信证据与开源数据之间的关系及可信证据与可信属性之间的关系，同时给出开源软件的可信属性的数学定义，在此基础上给出开源软件的可信性的定义[3]。

目前，越来越多的互联网公司通过缺陷跟踪系统管理产品在生命周期中遇到的缺陷，记录了从产生、确认、修复、测试通过、关闭、重新打开等各种状态下的缺陷。比较常用的缺陷管理工具有 Bugzilla 系统，Debbugs 系统及禅道系统等。通过缺陷工具获取可信证据，并给出可信证据与可信属性的关系：①无法修复的缺陷个数，度量开源软件的安全性；②已确认的缺陷个数，度量开源软件的可靠性；③缺陷平均修复代价，度量开源软件的防危性；④已修复的缺陷个数，度量开源软件的可维护性。

开源软件安全性的数学定义。

定义 1　令 F_k 为对开源软件某组件功能操作前排除“隐患”的度量[3]。让

$$Se=\begin{cases} f_0, F_k \geqslant f_0 \\ f_1, f_0 > F_k \geqslant f_1 \\ f_2, f_1 > F_k \geqslant f_2 \\ \cdots \\ f_n, f_{n-1} > F_k \geqslant f_n \end{cases} \tag{1}$$

则可对开源软件的安全性进行定义，记为：$P\{Se=f_k\}=PSe$。此定义的详细内容可参见文献[4]。

开源软件可靠性的定义。

定义 2　W_k 表示对开源软件某组件功能操作时错误发生数量的度量[3]。让

$$R=\begin{cases} w_0, W_k \leqslant w_0 \\ w_1, w_0 < W_k \leqslant w_1 \\ w_2, w_1 < W_k \leqslant w_2 \\ \cdots \\ w_n, w_{n-1} < W_k \leqslant w_n \end{cases} \tag{2}$$

则可对开源软件的可靠性进行定义，记为：$P\{R=w_k\}=PR$。此定义的详细内容可参见文献[4]。

开源软件防危性的定义。

定义 3　L_k 表示对开源软件某组件功能操作失败后代价的度量[3]。让

$$Sa=\begin{cases} l_0, L_k \leqslant l_0 \\ l_1, l_0 < L_k \leqslant l_1 \\ l_2, l_1 < L_k \leqslant l_2 \\ \cdots \\ l_n, l_{n-1} < L_k \leqslant l_n \end{cases} \tag{3}$$

则可对开源软件的防危性进行定义，记为：$P\{Sa=l_k\}=PSa$。此定义的详细内容可参见文献［4］。

开源软件可维性的定义。

定义 4　M_k 为开源软件某组件操作中可修复错误数量的比率[3]。让

$$M=\begin{cases} m_0, M_k \geqslant m_0 \\ m_1, m_0 > M_k \geqslant m_1 \\ m_2, m_1 > M_k \geqslant m_2 \\ \cdots \\ m_n, m_{n-1} > M_k \geqslant m_n \end{cases} \tag{4}$$

则可对开源软件的可维性进行定义，记为：$P\{M=m_k\}=PM$。此定义的详细内容可参见文献［4］。

定义 5　D_{ck} 为开源软件的可信性，让

$$D_{Ck}=\alpha_1\times P_{Se}+\alpha_2\times P_R+\alpha_3\times P_{Sa}+\alpha_4\times P_M \tag{5}$$

此定义的详细内容可参见文献［4］。

1.2　开源软件可信转移概率

文献［4］提出了 SOA 软件代数模型的可信软件范式，对满足 DNF 范式的 SOA 软件论证了满足半 Markov 过程的条件，从而利用 Markov 模型计算出 SOA 软件系统的可信性。开源软件是由若干个组件构成，任意一个组件都至少与其它组件中的一个发生组合，满足服务组件依赖第一范式；任意组件的组合产生的消息属激发运算组合，故开源软件所有的服务依赖为激发运算组合，满足 SOA 软件可信范式 DNF[3]。因此，开源软件也满足半 Markov 过程的条件。开源软件的可信性转移概率可由式（6）得

$$\begin{pmatrix} C_{1n} \\ C_{2n} \\ . \\ . \\ C_{nn} \end{pmatrix}=\begin{pmatrix} Q_{11} & Q_{12} & . & . & Q_{1n} \\ Q_{21} & Q_{22} & . & . & Q_{2n} \\ . & . & Q_{ij} & . & . \\ . & . & . & . & . \\ Q_{n1} & Q_{n2} & . & . & Q_{nn} \end{pmatrix}\times\begin{pmatrix} C_{1n} \\ C_{2n} \\ . \\ . \\ C_{nn} \end{pmatrix} \tag{6}$$

此公式的详细内容可参见文献［4］，其中：tij 为开源软件组件 i 到 j 的转移概率，oi(t) 为开源软件组件组合运算的可信性，di(t) 为开源软件组件的可信性。

2　开源软件可信性算法

获取缺陷系统中可信数据的可信算法具体如下：

算法 1：爬取数据

输入：生成 URL 文件

输出：URL 网页

(C)1994-2024 China Academic Journal Electronic Publishing House. All rights reserved.　http://www.cnki.net

缺陷跟踪系统中缺陷的 URL 列表被爬取出来，并保存于 URL 文件，通过 URL 文件逐一爬取 url 网页。

算法描述：

```
Procedure Crawler()：
    //生成 URL 文件
    Function urlFile()
     {
    //读取网页源代码
IF(file. getAttribute( "href")) THEN
{
    //保存 href 链接到集合
    set. add(file)；
/}
For urlfile：=set
{
    //将 URL 存入到 URL 文件
    Output(urlfile)；
/}
/}
Function preParse()
{
//读取 URL 文件
Reader in = new FileReader(urlfile)；
BufferedReaderreader=new BufferedReader(in)；
String line=null；
//初始化 URL 集合
While((line=reader. readLine()) ! =null) do {
        urlset. add(line)；
/}
//获取每一个 url 网页
For string：=urlset
{
    Getbug(string)；
/}
/}
```

算法 1 从 url 网页中获取的是原始数据，结合可信属性定义需提取可信证据项，过滤冗余数据，算法 2 则给出具体的解析过程。

算法 2：数据解析

输入：URL 网页

输出：可信属性数据

提取的可信证据项包括缺陷的 ID、status、reportedtime、modifiedtime、importance 等数据，提取的证据项保存到本地数据库。

算法描述：

```
Function parseHTML(inputstream，url)
{
    Get(bugid)；//获取缺陷 ID
    Get(status)；//获取缺陷状态
    Get(reportedtime)；//获取缺陷第一次提交时间
    Get(modifiedtime)；//获取缺陷最后一次修复时间
    Get(importance)；//获取缺陷级别
  bug=(status，reportedtime，modifiedtime，importance，bugid)；
        Save(bug)；//保存到数据库
/}
```

算法 2 提取了可信证据项，根据开源软件可信性模型中可信证据项与可信属性的映射关系，得出组件可信性及可信性转移概率，通过算法 3 可计算出可信性转移概率分布[3]。

算法 3：可信性转移概率分布计算

输入：组件可信性及可信性转移概率

输出：组件之间可信转移概率分布

算法描述：

```
Procedure DependentCalculate()：
//创建矩阵
Function createJuzhen(m)
{
    //创建一个 m 行(m+1) 列的增广矩阵
    Double[][]Q = new Double[m][m+1]；
    For i：=0 To m
      For j：=0 To m
{
  //从后台输入数据，并存放到 Q 数组
Q[i][j] =input. nextDouble()；
/}
/}
/*上三角变换（找出每列绝对值最大数，行变换，并消去)，n 为矩阵的行数，Q 为矩阵的指针*/
Function uptrbk(n，Q)
{
        Integer p，j，k；Double m=0；
For p：=0 To n-1；
//找出该列最大项的标号
        j = findMax(p，n，Q)；
//交换 p 行和 j 行
exchangeRow(p，j，n，Q)；
        //消除 p 元素以下的 p 列内容
For k：=p+1 To N
      {
m=Q [k] [p] /Q [p] [p]；
        For q：=p To N+1
```

(C)1994-2024 China Academic Journal Electronic Publishing House. All rights reserved. http://www.cnki.net

```
            Q[k][q] =Q[k][q] -m*Q [p][q];
            /}
    /}
    //回代函数
    Function backSub(Q, n)
    {
        //X[] 数组用来存储最后的计算结果
        double[] X = new double[N+1];
        //求出增广矩阵最后一个值
        X[N-1] =Q[(N-1)] [N] /Q[(N-1)] [N-1];
    //通过得出的最后一个结果依次往增广
    //矩阵中回代
        For k: =N-2 To 0
    {
        For i: =k+1 To N
          temp=temp+Q[k] [i] *X[i];
          X[k] =(Q[k] [N] -temp) /Q[k] [k];
        /}
    /}
```

3　案例研究

Firefox 是一款应用比较广泛且较为成熟的开源软件，其缺陷数据是通过 Bugzilla 系统进行管理，Bugzilla 系统记录了 Firefox 在不同时期不同组件出现的各种缺陷和状态。其中，缺陷的状态包括：unconfirmed、assigned、new、resolved、reopen、closed、verified、fixed、duplicate、worksforme、wontfix、invalid[3]。各状态可描述为：

unconfirmed：对提交的测试报告进行验证，验证未通过时(缺陷未能重现)，该缺陷被认为是不能被确认的。

assigned：缺陷被分配给不同的开发人员进行处理。

new：对提交的测试报告进行验证，验证通过时(缺陷重现)，该缺陷被认为是一个新的缺陷。

resolved：缺陷已经解决。

reopen：已修复的缺陷复现时，对该缺陷重新标记需要修复。

closed：缺陷被成功处理，并关闭。

verified：缺陷通过测试。

fixed：缺陷修复完成。

duplicate：新提交的缺陷被认定为与已提交的缺陷重复，并认定已提交缺陷的 ID 为新提交缺陷的 ID。

worksforme：缺陷通过测试不能复现。

wontfix：不能完成缺陷的修复。

invalid：通过测试认为不属于缺陷，测试报告认定为无效。

3.1　Firefox 浏览器组件之间联系的研究

Firefox 浏览器所包含的组件及其所代表的含义可参见文献 [3]。

Firefox 浏览器中的 63 个组件，从缺陷系统中获取的可信证据验证得出：组件之间的缺陷转移分布，如表 1 所示，即组件 M_2 中未修复的缺陷导致组件 M_1 产生了 43 个缺陷，组件 M_3 中未修复缺陷导致组件 M_1 产生了 30 个缺陷，组件 M_4 中未修复的缺陷导致组件 M_1 产生了 14 个缺陷，组件 M_5 中未修复的缺陷导致组件 M_1 产生了 18 个缺陷，…，组件 M_{60} 中未修复的缺陷导致组件 M_1 产生了 18 个缺陷等等。从表中可知，63 个组件之间的缺陷转移分布都存在类似的联系，由此可以得出结论：缺陷之间是有依赖关系的，每一个组件中的缺陷可能是因为不同组件中的缺陷而产生的。因此，组件与组件之间存在转移关系[3]。

表 1　组件缺陷转移分布

	M_1	M_2	M_3	M_4	M_5	…	M_{60}	M_{61}	M_{62}	M_{63}
M_1		43	30	14	18	…	18	0	0	0
M_2	322		270	137	627	…	260	31	67	10
M_3	69	103		7	47	…	22	0	1	4
M_4	69	104	621		47	…	22	0	1	4
M_5	53	159	35	20		…	177	1	11	3
…	…	…	…	…	…		…	…	…	…
M_{60}	41	175	26	29	179	…		2	0	3
M_{61}	0	80	0	0	4	…	2		0	0
M_{62}	0	6	0	0	4	…	0	0		0
M_{63}	3	35	3	3	4	…	5	0	0	

3.2　可信性评价

分析缺陷系统中获取的 Firefox 浏览器 63 个组件的可信证据与可信属性安全性、可靠性、防危性、可维性的映射关系可知，状态为 wontfix 缺陷个数可度量安全性，状态为 new 缺陷个数可度量可靠性，状态从 new 到 fixed 修复的平均时间代价可度量防危性，状态为 fixed 缺陷个数可度量可维性等。可信证据通过算法 1、算法 2 及式 (1) ～式 (4) 可得出组件的可信属性概率和可信值。图 1 为自动化

(C)1994-2024 China Academic Journal Electronic Publishing House. All rights reserved.　http://www.cnki.net

获取的 63 个组件的名称、缺陷总数、安全性缺陷个数、可靠性缺陷个数、可维护性缺陷个数、安全性概率、可靠性概率、可维性概率、防危性概率及可信性概率的结果，其中，防危性缺陷的个数与缺陷总个数相同。表 2 为 63 个组件安全性、可靠性、可维性、防危性及可信性整理后的数据；表 3 根据 Firefox 浏览器各个组件之间存在依赖关系，组件之间存在转移概率的结论而得到[3]。根据表 2 和表 3 计算可信转移概率分布见表 4。

以 M_1 到 M_{64} 的可信性为例：

D1，64 = 0.22 + 0.012D2，64 + 0.015D3，64 + 0.015D4，64+0.006D5，64

D2，64 = 0.04 + 0.009D1，64 + 0.019D3，64 + 0.019D4，64+0.015D5，64

D3，64 = 0.17 + 0.007D1，64 + 0.010D2，64 + 0.131D4，64+0.004D5，64

D4，64 = 0.15 + 0.003D1，64 + 0.005D2，64 + 0.002D3，64+0.002D5，64

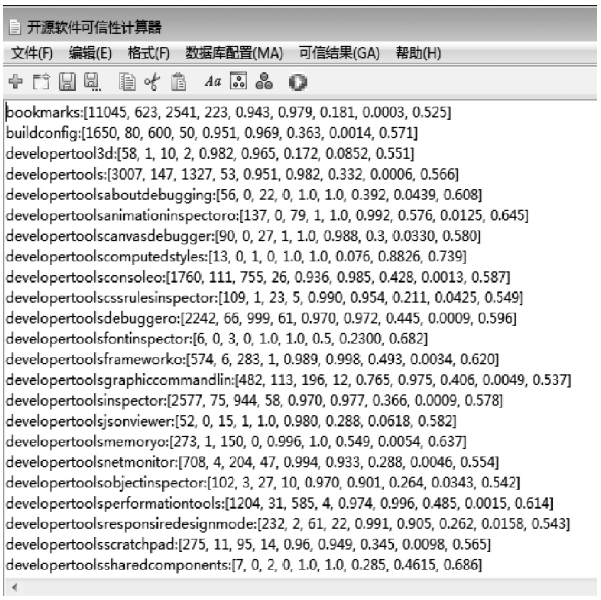

图 1 组件可信性计算结果

表 2 各组件可信属性值及可信结果

	M_1	M_2	M_3	M_4	M_5	…	M_{60}	M_{61}	M_{62}	M_{63}	L
安全性	0.943	0.990	0.944	0.932	0.947	…	0.945	0.962	0.946	0.982	0.958
可靠性	0.979	0.760	0.953	0.944	0.962	…	0.953	0.985	0.988	0.992	0.952
防危性	0.0003	0.0025	0.0013	0.0019	0.0005	…	0.0008	0.0118	0.009	0.0179	0.0005
可维性	0.181	0.044	0.167	0.152	0.250	…	0.193	0.369	0.642	0.005	0.095
可信性	0.525	0.449	0.516	0.507	0.539	…	0.522	0.581	0.646	0.499	0.501

表 3 各个组件转移概率

	M_1	M_2	M_3	M_4	M_5	…	M_{60}	M_{61}	M_{62}	M_{63}	M_{64}	F
M_1		0.044	0.056	0.056	0.022	…	0.016	0	0	0	0.22	0.1
M_2	0.039		0.084	0.085	0.067	…	0.036	0.004	0.009	0.001	0.04	0.02
M_3	0.027	0.037		0.507	0.015	…	0.018	0	0.001	0.003	0.17	0.07
M_4	0.013	0.019	0.006		0.008	…	0.018	0	0.001	0.003	0.15	0.09
M_5	0.016	0.086	0.038	0.038		…	0.075	0.001	0.005	0.001	0.40	0.07
…	…	…	…	…	…		…	…	…	…	…	…
M_{60}	0.027	0.116	0.017	0.019	0.119	…		0.001	0	0.002	0.23	0.07
M_{61}	0	0.336	0	0	0.017	…	0.008		0	0	0.33	0.07
M_{62}	0	0.042	0	0	0.028	…	0	0		0	0.60	0.11
M_{63}	0.012	0.139	0.012	0.012	0.016	…	0.02	0	0		0.005	0.02

表 4 各个组件可信转移概率分布

	M_1	M_2	M_3	M_4	M_5	…	M_{60}	M_{61}	M_{62}	M_{63}	M_{64}
M_1		0.012	0.015	0.015	0.006	…	0	0	0	0.058	0.22
M_2	0.009		0.019	0.019	0.015	…	0.008	0.001	0.002	0	0.04
M_3	0.007	0.010		0.131	0.004	…	0.005	0	0	0.001	0.17
M_4	0.003	0.005	0.002		0.002	…	0.005	0	0	0.001	0.15
M_5	0.004	0.023	0.010	0.010		…	0.020	0	0.001	0	0.40
…	…	…	…	…	…	…	…	…	…	…	…
M_{60}	0.007	0.030	0.004	0.005	0.031	…		0	0	0.001	0.23
M_{61}	0	0.098	0	0	0.005	…	0.002		0	0	0.33
M_{62}	0	0.014	0	0	0.009	…	0	0		0	0.60
M_{63}	0.003	0.035	0.003	0.003	0.004	…	0.005	0	0		0.005

(C)1994-2024 China Academic Journal Electronic Publishing House. All rights reserved. http://www.cnki.net

D5，64 = 0.40 + 0.004D1，64 + 0.023D2，64 + 0.010D3，64+0.010D4，64

解：M_1 到 M_{64} 的可信性结果为 0.228；M_2 到 M_{64} 的可信性结果为 0.055；M_3 到 M_{64} 的可信性结果为 0.194；M_4 到 M_{64} 的可信性结果为 0.152；M_5 到 M_{64} 的可信性结果为 0.406。由此可以求出任何一个组件转移到成功的可信性。

4　相关研究比较

从实际意义方面考虑。袁霖等提出基于互联网的开源软件自动化可信证据框架，分析了软件开发阶段人员特征和配置角色结构对软件可信的影响，其中以自动化获取的证据评估一个软件的可信性，但对自动化搜集的可信证据或证据项未能进行更精确的数学计算。本文的相关研究结果及 Firefox 案例研究对比结果表可参见文献［3］。

Firefox 案例研究结果见表 5。

对比已有的可信结果可知，本文能够得出 Firefox 在近 6 个月中安全性、可靠性、防危性、可维性及自身整体的可信结果值。除此之外，对于 Firefox 中组件的转移概率可以得出具体的估值，用户能够更直观的评估一个开源软件的可信性[3]。

表 5　Firefox 案例研究对比结果

可信属性/可信概率	可信证据	已提出的结论(个数)	本文结论(概率)
安全性	近 6 个月处于 open 状态的 P1/critical 级缺陷个数或概率	34	0.96
可靠性	近 6 个月被确认的缺陷个数或概率	72	0.15
防危性	近 6 个月缺陷平均修复时间或平均修复概率	3(周)	0.0006
可维性	近 6 个月被修复的缺陷个数或概率	82	0.95
Firefox 可信概率	对安全性、可靠性、防危性及可维性整体的评估	无	0.51
组件转移成功的概率	组件中存在的缺陷之间的联系	无	可计算

本文的工作量主要体现在自动化获取可信证据和计算 Qij(t) = Pij * di(t)。其中，采用爬虫技术获取可信证据，公式 Qij(t) = Pij * di(t) 中的 Pij 可以通过测试工具获得的可信属性数据计算；而 di(t) 是开源软件中各个组件的可信性概率。因此，开源软件可信性评估方法具有可操作性[3]。

5　结束语

本文针对开源软件可信性研究，提出开源软件可信性模型，并对开源软件的组件之间的依赖关系和转移概率给出了相应的结论。并通过对 Firefox 案例研究，能够获得准确的可信性评估数值。

本文仍存在一些缺陷，对于开源软件可信性评估只从缺陷跟踪系统的角度进行数学化定义与计算，不够全面。下一步工作中，可对邮件交互系统及 Web 页面进行可信性数学化评估。

参考文献：

［1］CHEN Huowang，WANG Ji，DONG Wei. High confidence software engineering technologies［J］. ACTA Electronica Sinica，2003，31（12A）：1934-1938（in Chinese）.［陈火旺，王戟，董威. 高可信软件工程技术［J］. 电子学报，2003，31（12A）：1934-1938.］

［2］YUAN Lin，WANG Huaimin，YIN Gang，et al. Automatic evaluation evidence framework for open source software［J］. Computer Engineering & Science，2013，35（2）：138-141（in Chinese）.［袁霖，王怀民，尹刚，等. 开源软件自动化评估证据框架［J］. 计算机工程与科学，2013，35（2）：138-141.］

［3］LIU Lili. Design and implementation of web system dependability evaluation tool［D］. Beijing：North China University of Technology，2017：1-44（in Chinese）.［刘丽丽. Web 系统可信性评价工具的设计与实现［D］. 北京：北方工业大学，2017：1-44.］

［4］ZHAO Huiqun，SUN Jing. A methodological study of evaluating the dependability of SOA software system［J］. Chinese Journal of Computers，2010，33（11）：2203-2210（in Chinese）.［赵会群，孙晶. 一种 SOA 软件系统可信性评价方法研究［J］. 计算机学报，2010，33（11）：2203-2210.］

［5］MENG Xianying. A brief analysis on software defect management and tool［J］. China Academic Journal Electronic Publishing House，2011，24（10）：77-78（in Chinese）.［孟显英. 浅析软件缺陷管理及工具的选择［J］. 电脑开发与应用，2011，24（10）：77-78.］

［6］ZHAO Qian，WANG Huiqiang，FENG Guangsheng，et al. Measuring method of software dependability based on Pi calculus［J］. Journal of Jilin University（Engineering and Technology Edition），2011，41（6）：1685-1689（in Chinese）.［赵倩，王慧强，冯光升，等. 基于 Pi 演算的软件可信性度量方法［J］. 吉林大学学报（工学版），2011，41（6），1685-1689.］

［7］FU Yanggeng，GONG Xiaoting，WU Yingjie. An evidential reasoning approach for trustworthy software evaluation under systems development life cycle［J］. Journal of Nanjing Normal University（Natural Science Edition），2013，36（1）：133-141（in Chinese）.［傅仰耿，巩晓婷，吴英杰. 基于证据推理面向生命周期的可信软件评估［J］. 南京师大学报（自然科学版），2013，36（1）：133-141.］

［8］DU Jing，YANG Ye，WANG Qing，et al. Evidence-based

(C)1994-2024 China Academic Journal Electronic Publishing House. All rights reserved.　http://www.cnki.net

trustworthy software process assessment method [J]. Journal Frontiers of Computer Science and Technology, 2011, 5 (6): 502-512 (in Chinese). [杜晶，杨叶，王青，等. 基于证据的可信软件过程评估方法 [J]. 计算机科学与探索，2011，5 (6)：502-512.]

[9] WANG Huanhuan, ZHAN Yongzhao, CHEN Jinfu. Research on trust worthy software analysis and testing [J]. Application Research of Computer, 2011, 28 (7): 2402-2406 (in Chinese). [王环环，詹永照，陈锦富. 可信软件分析与测试研究进展 [J]. 计算机应用研究，2011，28 (7)：2402-2406.]

[10] CHEN Si, WANG Shuyan, SUN Jiaze. Trusted software reliability measures based on cloud model [J]. Application Research of Computers, 2014, 31 (9): 2729-2732 (in Chinese). [陈思，王曙燕，孙家泽. 基于云模型的可信软件可靠性度量模型 [J]. 计算机应用研究，2014，31 (9)：2729-2732.]

[11] Salvatore Distefano. Dependability assessment of critical systems [J]. J Ambient Intell Human Computer, 2015, 6: 713-720.

[12] Claudio A Ardagna, Ravi Jhawar, Vincenzo Piuri. Dependability certification of services: A model-based approach [J]. 2015, 97 (1): 51-78.

[13] SHEN Guohua, HUANG Zhiqiu, XIE Bing, et al. Survey on software trustworthiness evaluation: Standards, models and tools [J]. Journal of Software, 2016, 27 (4): 955-968 (in Chinese). [沈国华，黄志球，谢冰，等. 软件可信评估研究综述：标准、模型与工具 [J]. 软件学报，2016，27 (4)：955-968.]

[14] ZHOU Jian, ZHANG Mingxin. Survey on Trustworthy software evaluation [J]. Application Research of Computers, 2012, 29 (10): 3610-3613 (in Chinese). [周剑，张明新. 软件可信评估综述 [J]. 计算机应用研究，2012，29 (10)：3610-3613.]

[15] LUO Xinxing, TANG Zhenyu, ZHAO Yujie. Dynamic software reliability assessment based on Markov chain [J]. Application Research of Computers, 2015, 32 (8): 2401-2405 (in Chinese). [罗新星，唐振宇，赵玉洁. 基于马尔科夫链的可信软件动态评估模型 [J]. 计算机应用研究，2015，32 (8)：2401-2405.]

[16] LI Xiaohong, WANG Xiangyu, ZHANG Tao, et al. Software trustworthiness evaluation based on weakness analysis and testing assessment [J]. J Tsinghua University (Science & Technology), 2011, 51 (10): 1288-1293 (in Chinese). [李晓红，王翔宇，张涛，等. 基于缺陷分析与测试评审的软件可信性评价方法 [J]. 清华大学学报（自然科学版），2011，51 (10)：1288-1293.]

（上接第 3271 页）

[8] Meng Xinbin, Gao XZ, Lu LH. A new bio-inspired optimization algorithm: Bird swarm algorithm [J]. Journal of Experimental and Theoretical Artificial Intelligent, 2015, 28 (4): 1-15.

[9] WU Husheng, ZHANG Fengming, WU Lushan. A new swarm intelligence algorithm: Wolf pack algorithm [J]. Systems Engineering and Electronics, 2013, 11 (35): 2430-2438.

[10] Ahmed Ibrahem Hafez, Hossam M Zawbaa, Emary E, et al. An innovative approach for feature selection based on chicken swarm optimization [C] //Seventh International Conference of Soft Computing and Pattern Recognition. IEEE, 2015: 19-24.

[11] MENG Xianbing, LIU Yu, GAO Xiao, et al. A new bio-inspired algorithm: chicken swarm optimization [G]. LNCS 8794: ICSI, 2014: 86-94.

[12] LI Guoliang, WEI Zhenghua, XU Lei. Wolf pack algorithm based on modified search strategy [J]. Journal of Computer Applications, 2015, 35 (6): 1633-1636 (in Chinese). [李国亮，魏振华，徐蕾. 基于改进搜索策略的狼群算法 [J]. 计算机应用，2015，35 (6)：1633-1636.]

(C)1994-2024 China Academic Journal Electronic Publishing House. All rights reserved. http://www.cnki.net